

COMP3222 – Machine Learning Technologies

Lab Sheet 2 (Week 3)

When: Monday 5th October 2026, 13:00–15:00.

Based on: Week 2 lectures: Decision Trees. This lab sheet reinforces low-level understanding of split criteria, top-level usage of decision trees for understanding data and implementation-level details of how scikit-learn implements decision trees.

There are three tasks:

- Task 1: Implement a Gini impurity and Gini gain split quality measure
- Task 2: Explore using decision tree classification in scikit-learn
- Task 3: Understand the decision tree implementations in scikit-learn

Please base your exercises on the code in the course lab repository:

GitHub: <https://github.com/TonyBagnall/COMP3222-2026>

Task 1: Implement the Gini gain split quality measure

Your first task is to implement simple Python functions to find the Gini impurity and Gini gain, then test them using the toy example we used in the lecture. We used the play golf example in the lectures, and you can get this file from Moodle or the GitHub repo.

```
data = pd.read_csv(FULL_PATH)
y = data.iloc[:, 4].to_numpy()
X = data.iloc[:, 0:4].to_numpy()
```

The Gini value for a single node is given in lecture 2.2. Re-read this section before this task. For a single node, the impurity is

$$I(Z) = 1 - \sum_{i=1}^c p(i)^2$$

A) Implement a simple function to find the impurity from counts

```
def impurity(counts: np.ndarray) -> float:
    """Return Gini impurity for one node given class
    counts."""
```

Find the counts for the array **y** (use **np.unique**) and verify the impurity is the same as in the notes.

B) Implement a function to find the Gini gain for a given attribute

The Gini gain for an attribute is

$$\text{Gini}(Z, A) = I(Z) - \sum_{W \in V} \frac{|W|}{|Z|} I(W)$$

Now implement a function to work out the Gini gain from splitting on a categorical attribute. You may assume that the attribute is encoded as integers.

```
def gini_gain(attr: np.ndarray, y: np.ndarray) -> float:
    """Compute Gini gain for splitting labels y by categorical
    attribute attr."""
```

This function should

1. **Find the impurity of the root:** find counts of **y**, call the function **impurity** defined above. ($I(Z)$ in the equation)
2. **Split y by the values of attr and find impurity of each split**
the terms $I(W)$ in the equation

3. Calculate and return the Gini gain ($\text{Gini}(Z, A)$)

Working through the example in the lecture will help. You can implement this in C style with loops and Numba, or with NumPy operations using

`np.unique(attr, return_inverse=True).`

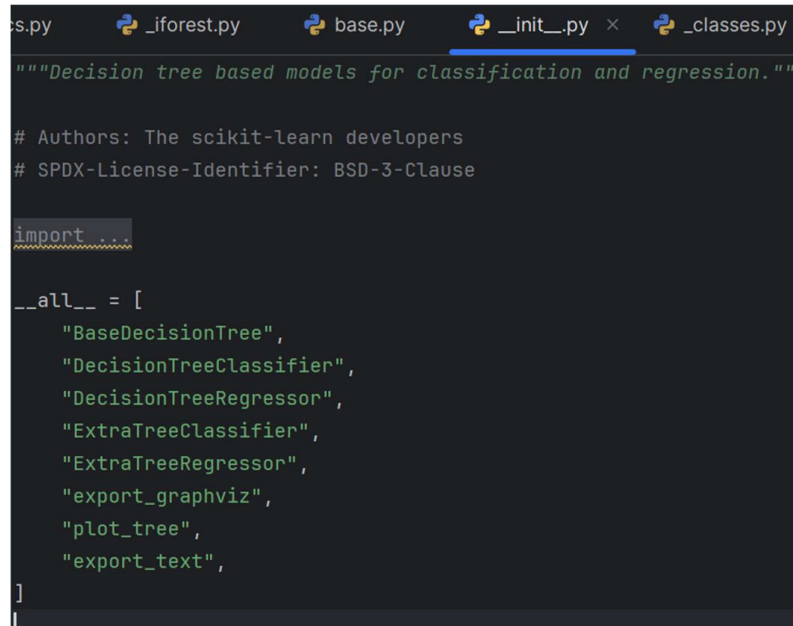
It is a bit fiddly; if you get stuck, move on to Task 2 and revisit it later.

Finally, work out the Gini gain for each attribute in play golf and verify it is the same as the demo/lecture (you will need to convert Outlook to integers).

Task 2: decision tree classification in scikit-learn

Scikit-learn has limited decision tree functionality, primarily based on CART. Read the documentation:

<https://scikit-learn.org/stable/modules/tree.html>

A screenshot of a code editor with several tabs at the top: 'cs.py', '_iforest.py', 'base.py', '_init_.py' (which is selected and highlighted in blue), and '_classes.py'. The code in the '_init_.py' file is as follows:

```
"""Decision tree based models for classification and regression."""

# Authors: The scikit-learn developers
# SPDX-License-Identifier: BSD-3-Clause

import ...

__all__ = [
    "BaseDecisionTree",
    "DecisionTreeClassifier",
    "DecisionTreeRegressor",
    "ExtraTreeClassifier",
    "ExtraTreeRegressor",
    "export_graphviz",
    "plot_tree",
    "export_text",
]
```

(We cover regression next week; focus on classification here.)

DecisionTreeClassifier is based on CART and does not work with categorical data.

ExtraTreeClassifier is a variant of extremely randomized trees

[<https://link.springer.com/article/10.1007/s10994-006-6226-1>] which we don't cover but which are often used as components for ensembles.

We cover the parameters in the next task; this task is about using these classifiers and visualising the output using **plot_tree**. The exercise is similar to those in Chapter 6 of HOML (page 195 onwards), but using our play golf example.

A) Formatting

Try fitting a **DecisionTreeClassifier** on the play golf data from Task 1. It should raise an error: *"ValueError: could not convert string to float"*. One-hot encode Outlook to form **X** with shape (14, 6); y has shape (14,). You can do this manually by using **OneHotEncoder** and reforming X, but it is better to use the **ColumnTransformer** in conjunction with the **OneHotEncoder** in a **Pipeline**.

B) Fit and visualise a decision tree classifier

Scikit-learn has a nice built-in visualisation for trees using matplotlib.

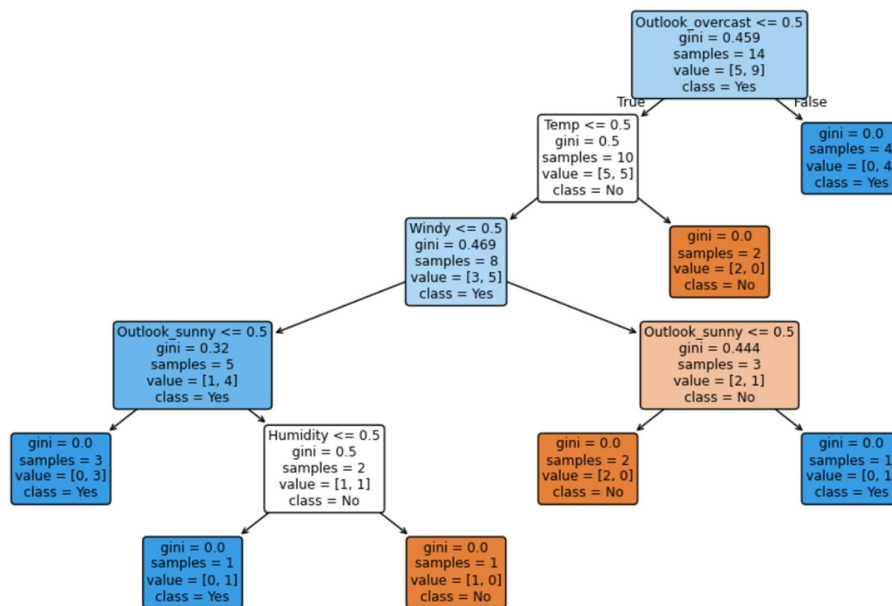
```
import matplotlib.pyplot as plt
from sklearn import tree
```

You need to form a list of names for the attributes and the classes, fit a classifier, then do something like this:

```
dt = DecisionTreeClassifier()
dt.fit(X,y)

tree.plot_tree(
    dt,
    feature_names=feature_names,
    class_names=class_names,
)
plt.tight_layout()
plt.show()
```

You can make it look nicer (coloured, etc.) using arguments to `plot_tree` and configure the size with `plt` settings. It will only work with a subclass of `sklearn.tree._classes.DecisionTreeClassifier` or `sklearn.tree._classes.DecisionTreeRegressor`. You should see something like this:



Contrast it with the tree we found in the lecture. It is not the same: it is deeper (because of one-hot encoding). Which do you think is more interpretable? Try again using `ExtraTreeClassifier`. It also creates a completely different tree. All three classify the training data 100% accurately. How could we choose between them? More on evaluation in week 4.

You can also output the plot and load it into notebooks. See the examples in HOML. I have provided a function for making plots such as Figure 6.2 (page 198) in the file `lab2_task3`.

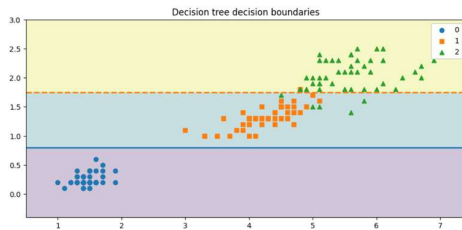
```

from lab2_task3 import plot_tree_2d
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier
iris = load_iris(as_frame=True)
X_iris = iris.data[["petal length (cm)", "petal width (cm)"]].values
y_iris = iris.target
tree_clf = DecisionTreeClassifier(max_depth=2, random_state=49)
tree_clf.fit(X_iris, y_iris)
plot_tree_2d(tree_clf, X_iris, y_iris)

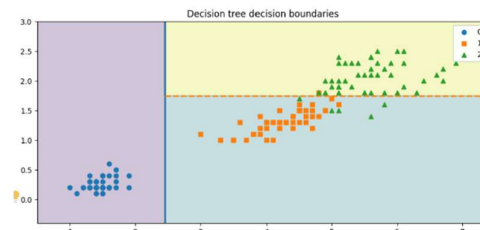
```

You may not get the same boundaries they display in the book (see below).

Random seed = 0



Random seed = 49



This randomness is caused by ties being settled randomly. **DecisionTreeClassifier** breaks equal-gain ties at random and randomly permutes features at each node; set **random_state** for reproducible trees.

Task 3: Inside **DecisionTreeClassifier**

Read the API documentation:

<https://scikit-learn.org/stable/modules/generated/sklearn.tree.DecisionTreeClassifier.html>

The goal of this task is to understand the decision tree parameters. Accompanying reading is Chapter 6 of HOML.

A) Design hyperparameters

criterion : {"gini", "entropy", "log_loss"}, default="gini"

The function to measure the quality of a split. Supported criteria are "gini" for the Gini impurity and "log_loss" and "entropy" both for the Shannon information gain,

splitter : {"best", "random"}, default="best"

The strategy used to choose the split at each node. Supported strategies are "best" to choose the best split and "random" to choose the best random split.

class_weight : dict, list of dict or "balanced", default=None

Weights associated with classes in the form ``{class\_label: weight}`` .

“entropy” and “log_loss” are identical and equivalent to Information Gain, covered in Lecture 2 from slide 23. I have no idea why they overload the name.

The splitter parameter determines how split values are found on continuous attributes. “best” means try all values, “random” means it samples random thresholds per considered feature and picks the best among those considered.

Try different settings on play golf and see if there is any effect on the final tree. I doubt there will be on this toy problem.

B) Regularisation hyperparameters

For this part we will use a bigger dataset with only continuous attributes. The ItalyPowerDemand data is a classic in the time series field.

<https://www.timeseriesclassification.com/description.php?Dataset=ItalyPowerDemand>

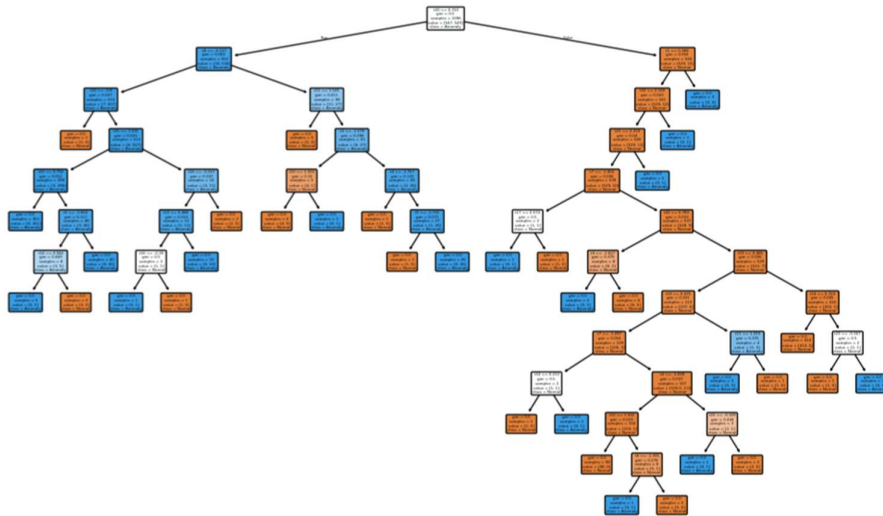
```
from aeon.datasets import load_italy_power_demand
X, y = load_italy_power_demand(return_type="numpy2D")
y = y.astype(np.int64)
```

More generally, any problem from the UCR/UEA archives can be loaded by name with load_classification, which returns a 3D array (cases, channels, length):

```
from aeon.datasets import load_classification
X, y = load_classification("ItalyPowerDemand")
X = X.squeeze(axis=1) # (n_cases, 1, 24) -> (n_cases, 24)
y = y.astype(np.int64)
print(X.shape, y.shape)
```

Each case is a time series of length 24 of hourly power usage in Italy and there are 1096 cases. The problem is to predict if the day is in October to March (inclusive) [Class 1] or from April to September [Class 2] (the class variable loads as a string so we need to convert it to an int).

Train a decision tree on the Italy power demand data and plot the tree. It creates a complex tree that is probably overfitting, something like this:



This example tree has depth 12 and 26 leaf nodes. The fact there are more cases and the attributes are continuous means it will keep building until each leaf node is all of one class (pure). It could well be overfitting the data.

DecisionTreeClassifier has a range of hyperparameters that can be used to regularise the tree (reduce its complexity). These are listed below and control the early stopping of the tree. Sometimes these are called **pre-pruning parameters**.

max_depth : int, default=None

The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.

min_samples_split : int or float, default=2

The minimum number of samples required to split an internal node.

min_samples_leaf : int or float, default=1

The minimum number of samples required to be at a leaf node.

min_weight_fraction_leaf : float, default=0.0

min_impurity_decrease : float, default=0.0

A node will be split if this split induces a decrease of the impurity greater than or equal to this value.

max_leaf_nodes : int, default=None

Grow a tree with ``max\_leaf\_nodes`` in best-first fashion.

Explore the effect of setting these parameters on the decision tree structure for ItalyPowerDemand. An alternative to early stopping is to build a big tree then prune it.

DecisionTreeClassifier also has the ability to **post-prune** the tree. This is achieved through setting the parameter **ccp_alpha**. Try setting it to 0.1 on ItalyPowerDemand and see what happens to the tree. Then set it to 0.02. Read more about how it works here:

https://scikit-learn.org/stable/auto_examples/tree/plot_cost_complexity_pruning.html

How do we choose which to use? This is a huge topic in its own right, which we cover in detail in week 4.